

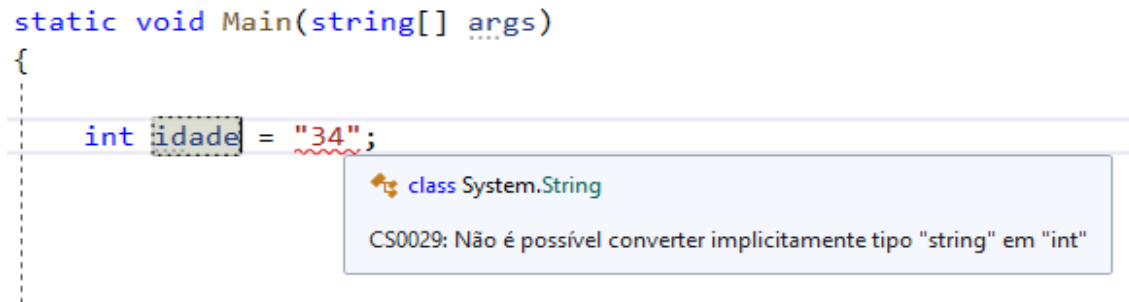
Conversão de Tipos de Dados em C#

Mário Leite

Originalmente, C# é uma linguagem com tipagem forte; embora possa trabalhar com tipagem dinâmica em situações especiais e convenientes para o programador; entretanto, mesmo partindo de uma variável tipada ela pode ser convertida para outro tipo de dado. Isto pode ser feito através da operação de **type casting** ou com a utilização de métodos da classe **Convert**.

Observe a instrução de declaração/iniciação nos esquemas, abaixo.

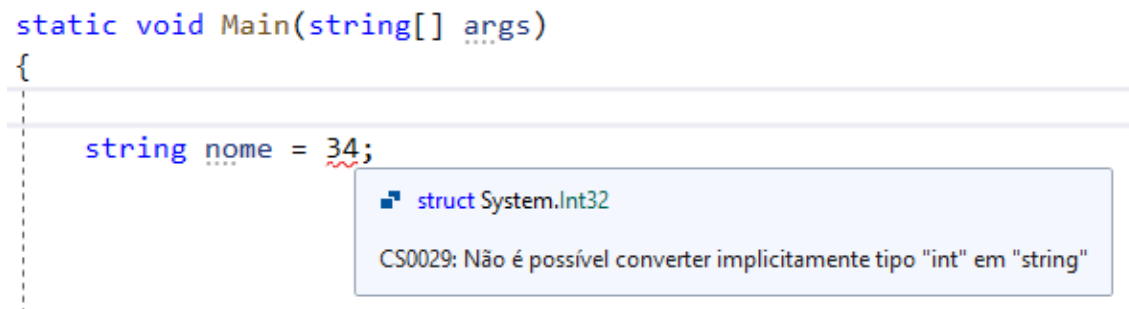
```
static void Main(string[] args)
{
    int idade = "34";
}
```



No esquema acima ocorre um erro na declaração da variável *idade*, pois o tipo de dado na declaração é **int** (*inteiro*) e seu valor iniciado é **string** (texto); portanto, a conversão *implícita* não é possível.

Agora, observe outra declaração...

```
static void Main(string[] args)
{
    string nome = 34;
}
```



Note o mesmo tipo de erro que ocorreu: incompatibilidade de tipos. A variável **nome** foi declarada como **string**, mas seu valor inicializado é do tipo **int**. Nos dois exemplos o tipo de erro é o mesmo: gerado como erro **CS0029**.

Obviamente, quando se declara uma variável com tipagem forte o programador deve iniciar com um valor compatível com o tipo declarado, mas se houver necessidade de alterar o tipo após essa declaração existem maneiras de fazer a conversão correta para evitar o erro mostrado acima.

Type Casting

É uma operação que permite fazer conversões de um tipo de dado para outro e que pode ser aplicado aos dois exemplos mostrados acima. Existem dois tipos de *type casting*: implícito e explícito.

1. Type Casting Implícito

Neste caso, a conversão é feita automaticamente ao passar de um tipo de tamanho menor para um tipo de tamanho maior. Deste modo o esquema de conversão obedece ao seguinte fluxo:

char -> int -> long -> float -> double

Observe o exemplo abaixo na declaração

```
internal class Program
{
    static void Main(string[] args)
    {
        int idade = '3';
    }
}
```

Observe que agora não ocorre o erro de incompatibilidade de tipos, pois foi feito um *type casting* implícito de *char* para *int*, e a variável *idade* passa a receber valores do tipo *int* como no código abaixo...

```
internal class Program
{
    static void Main(string[] args)
    {
        int idade = '3';
        idade = idade + 5;
        Console.WriteLine();
        Console.WriteLine(idade);
        Console.WriteLine("Tipo da variável idade: {0}", idade.GetType());
        Console.ReadKey();
    }
}
```

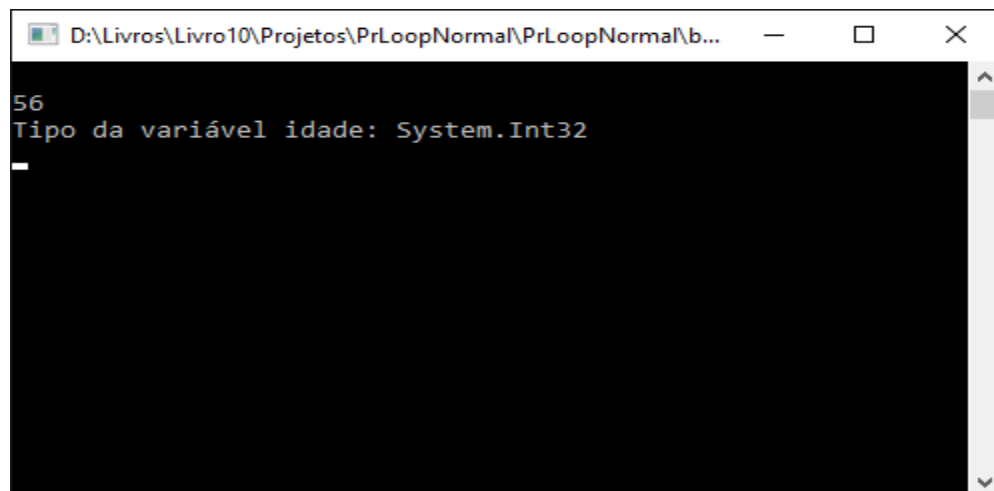


Figura 1 - Fazendo *type casting* implícito de *char* para *int*

Na **figura 1** o valor da variável **idade**, que inicialmente era **'3'** (*char*), ao ser incrementada de **5** resultou em **56**. Isso ocorre porque o caractere **'3'** possui o código Unicode **51**. Ao somar **5**, obtém-se **56**, que é o código do caractere **'8'**. Portanto, o cálculo foi feito com os códigos numéricos dos caracteres e não com os valores numéricos representados por eles.

Outra observação importante é o fato de a variável **idade** ser iniciada com apenas um caractere (**'3'** e não **'34'**, por exemplo), pois um **char** deve conter apenas um único caractere. Se fosse **'34'**, ocorreria erro (CS1012), pois a partir de dois caracteres o valor passa a ser considerado **string**.

2. Conversão explícita (explicit casting)

```
double x = 9.8;  
int y = (int)x;    //9
```

3. Conversão com métodos string s = "25"

```
int idade = Convert.ToInt32(s);
```

4. Conversão com parse

```
int x = int.Parse("123");
```

5. Conversão segura com TryParse

```
if (int.TryParse("123", out int valor))  
{  
    Console.WriteLine(valor);  
}
```
